DEPARTMENT OF MATHEMATICS & COMPUTER SCIENCE

Faculty of Science and Technology · National University of Lesotho

CS3520

Computer Organisation and Architecture I

Week 2 — Measuring & Analysing Computer Performance

Mr K. O. Setetemela

Course Coordinator · Department of Mathematics & Computer Science

Academic Year 2026/2027 · Semester A

RECAP & THIS WEEK

From Abstraction to Measurement

Last week: we traced a program from C, through the compiler and assembler, to the bits a RISC-V processor executes.

1	**Defining Performance**	Response time, throughput, and “n times as fast”
2	**The CPU Performance Equation**	Instructions × CPI × Clock Cycle Time
3	**Benchmarking**	Why we need standardised workloads
4	**Amdahl's Law**	The limits of optimising one part of a system
5	**Power & the Power Wall**	Why energy became the design constraint

MOTIVATION

Why Quantify Performance?

Three of our eight great ideas — parallelism, pipelining, prediction — are all performance techniques. To use them well as engineers, we need to measure performance rigorously, not just say “it feels faster.”



Compare Designs

Which processor, which algorithm, which compiler flag is actually better — and by how much?



Guide Engineering Trade-offs

Every design decision costs time, money, or power. Measurement tells you if it was worth it.



Communicate Honestly

Marketing claims (“2x faster!”) need a precise, defensible basis — or they are meaningless.

FIRST PRINCIPLES

Response Time vs Throughput

Response Time

(a.k.a. execution time)

Total time to complete one task, start to finish. What an individual user experiences.

Throughput

(a.k.a. bandwidth)

Total amount of work done per unit time. What matters for a shared, busy system.

If X completes a task faster than Y, we say X is “n times as fast” as Y:

```
Performance = 1 / Execution Time
            → PerformanceX / PerformanceY
            = ExecutionTimeY / ExecutionTimeX
            = n
```

WORKED EXAMPLE

How Much Faster, Exactly?

Computer A runs a program in 10 seconds. Computer B runs the same program in 15 seconds. How much faster is A than B?

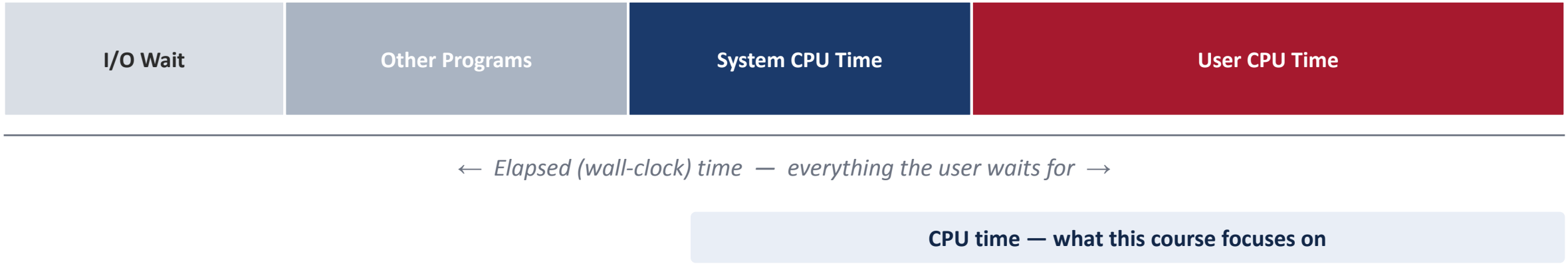



```
PerformanceA / PerformanceB  =  ExecutionTimeB / ExecutionTimeA
                             =  15 / 10
                             =  1.5
```

A is 1.5 times as fast as B — equivalently, B is 1.5 times slower than A.

Elapsed Time vs CPU Time

A shared computer runs many things at once — so “time” needs to be more precise than a stopwatch.



CPU time

The time the CPU actually spends computing your task — split into user CPU time (your program) and system CPU time (OS on your behalf).

This is what the CPU performance equation measures.

THE CORE FORMULA

The CPU Performance Equation

Instruction Count

How many instructions the program executes

× CPI

Average clock cycles per instruction

× Clock Cycle Time

Seconds per clock cycle (1 / clock rate)

```
CPU Time  =  Instruction Count  ×  CPI  ×  Clock Cycle Time
```

Three levers, one trade-off: a change to the instruction set, the microarchitecture, or the clock rate can move all three at once — rarely just one.

WORKED EXAMPLE

More Instructions ≠ Slower

Two RISC-V code sequences solve the same problem. Sequence 2 executes one more instruction — but is it slower?

Sequence	Instructions	Avg. CPI	Total Cycles
Sequence 1	5	2.0	**10 cycles**
Sequence 2	6	1.5	**9 cycles**

Sequence 2 executes more instructions but has a lower average CPI — fewer total cycles, so it finishes faster.

Instruction count alone never tells the whole story.

STANDARDISED MEASUREMENT

Benchmarking

Individual programs vary wildly. Benchmarks are agreed-upon workloads that let us compare systems fairly.

Real Programs	Actual applications you or others will run — the gold standard, but hard to compare across labs.
Kernels & Toy Benchmarks	Small, extracted code fragments (e.g. a matrix multiply) — easy to run, easy to misuse.
Synthetic / Industry-Standard Suites	e.g. SPEC — curated program collections designed for fair, repeatable, cross-vendor comparison.

We will benchmark our own RISC-V programs in Lab 1 this week using RARS's cycle counter.

THE LIMITS OF OPTIMISATION

Amdahl's Law

A rule stating that the speed-up possible from an improvement is limited by how much of the total task it actually affects.

$$\text{Execution Time}_{\text{After}} = \text{Execution Time}_{\text{Affected}} \div \text{Amount of Improvement} + \text{Execution Time}_{\text{Unaffected}}$$



Even an infinitely fast improvement to the 80% portion can only ever speed the whole task up by, at most, 5× — the untouched 20% becomes the new floor.

This is the law of diminishing returns, made precise.

WORKED EXAMPLE

Accelerating an AI Workload

An inference workload spends 80 of its 100 seconds in matrix-multiply. A new accelerator promises to speed multiply up dramatically. To make the whole workload 5× faster — how fast must the accelerator be?

1. `Target total time: 100s ÷ 5 = 20 seconds`
2. `Unaffected portion alone: 100 − 80 = 20 seconds`
3. `Required affected-portion time: 20 − 20 = 0 seconds`

No finite speed-up of the accelerator gets us there.

The 20-second unaffected portion caps the best possible outcome at 5×. If we truly need more, we must also attack the other 20% — the great idea of parallelism has limits set by what is left un-parallelised.

THE OTHER CONSTRAINT

Power, Energy & the “Power Wall”

Why Power Became the Limit

Faster clocks and denser transistors mean more heat. Past a point, chips cannot be cooled fast enough — this is the power wall that pushed the industry toward multicore.

Why It Matters for Lesotho

Energy-efficient computing lowers operating cost for national digital infrastructure — directly relevant as we build local AI and data-centre capacity.

THINK ABOUT IT

A vendor claims their new chip is “2× faster.”

Before you believe it: what workload was measured? Response time or throughput? Compared to which baseline? And does Amdahl’s Law limit the claim once it is applied to a real, mixed program?

Discuss with a neighbour for two minutes before we continue.

HANDS-ON THIS WEEK

Lab 1 & Tutorial Set 1

Lab 1 — Performance Measurement

- Run RISC-V programs in RARS/Ripes and read the instruction/cycle counters
- Compute CPI for two implementations of the same task
- Compare your measurements with a lab partner’s

Tutorial Set 1 — Issued This Week

- Performance-equation and Amdahl’s Law problems
- Not collected for marks — discussed in lecture/lab
- Use any resource, including AI tools, to practise

CHECKPOINT · CLO1

Key Takeaways

1. Distinguish response time from throughput, and compute relative performance precisely.
2. Apply the CPU performance equation (Instructions × CPI × Clock Cycle Time) to compare designs.
3. Use Amdahl’s Law to judge the real ceiling on any proposed optimisation.

COMING UP — WEEK 3

The RISC-V Instruction Set Architecture I

Instruction classes, formats & encoding — Lab 2: your first assembly programs

